

AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

Wydział Elektrotechniki, Automatyki, Informatyki i Inżynierii Biomedycznej

Projekt dyplomowy

Industrial Robotic Arm Supporting Assembly Tasks
Ramię robota przemysłowego wspomagającego zadania montażowe

Autor: *Piotr Zając*
Kierunek studiów: *Automatyka i Robotyka*
Opiekun pracy: *dr inż. Dawid Knapik*

Kraków, 2026

Contents

1. Introduction 9

1.1. Main goals of the project 11

1.2. Structure of the thesis 12

2. Theoretical background 13

2.1. ROS 2 framework 13

2.1.1. ROS 1 vs ROS 2 13

2.2. Basics of ROS 2 14

2.2.1. Nodes 14

2.2.2. Communication interfaces 14

2.3. URDF and SRDF 14

2.4. Large community projects: RViz, MoveIt 2, Micro-ROS 15

2.4.1. RViz 15

2.4.2. MoveIt 2 15

2.4.3. Micro-ROS 16

3. Mechanical design of the robot 17

3.1. Kinematic structure and degree-of-freedom selection 17

3.1.1. Degrees of freedom 17

3.1.2. Final selected layout 18

3.2. Actuator selection and torque calculations 18

3.3. Material selection and manufacturing methods 20

3.3.1. Manufacturing approach 20

3.3.2. Material comparison 20

3.3.3. Inserts, tolerances, and mechanical joints 21

3.3.4. Slicer settings and print orientation 22

3.4. Base (first joint) mechanical construction 24

3.4.1. Structural design 24

3.4.2. Electronics integration 24

3.4.3. First axis rotation mechanism 24

3.5. Arm structure 25
3.5.1. Shoulder and elbow joint (second and third axis) 25
3.5.2. Wrist (axes 4-6) 26
3.5.3. Cable management strategy 26
3.6. End effector 28
3.6.1. TCP reference tool 28
3.7. Assembly, tolerances, and iteration process 29
3.8. Summary of mechanical design 29
4. Electrical design of the robot 31
4.1. System architecture overview 31
4.1.1. ESP32 microcontroller 32
4.1.2. Micro-ROS integration 32
4.2. Actuator control 32
4.2.1. Stepper motor and DRV8825 driver 33
4.2.2. Hall sensor for homing 33
4.2.3. Servomotors 33
4.2.4. PCA9685 PWM driver 34
4.3. Power distribution and voltage regulation 34
4.3.1. Primary power supply 34
4.3.2. Voltage rails and regulation 34
4.3.3. Current consumption estimation 35
4.4. Summary of electrical design 37
5. Programming the Robot with ROS 2 39
5.1. Software architecture overview 39
5.2. ROS 2 environment 39
5.3. Robot description and visualization 39
5.3.1. URDF creation 40
5.3.2. RViz visualization 41
5.4. Motion planning with MoveIt 42
5.4.1. SRDF and MoveIt Setup Assistant 42
5.4.2. Trajectory planning with MoveIt 45
5.5. Hardware interface and Micro-ROS 46
5.5.1. Servomotors control via PCA9685 46
5.5.2. Stepper motor control and homing 46
5.5.3. Integrating hardware interface with Micro-ROS 47

5.5.4. Data exchange and verification 47
5.5.5. MoveIt–Micro-ROS bridge node 48
5.6. Advanced control: voice interface 48
5.6.1. C++ API interface for MoveIt 49
5.6.2. Voice recognition using the Vosk library 49
5.7. Software summary 50
6. Conclusion 51

1. Introduction

In recent years, access to advanced engineering knowledge and tools has become increasingly available. Free online educational resources have flattened the learning curve, and affordable electronic components and manufacturing tools, such as 3D printers, have significantly lowered the barrier to transforming engineering ideas into functioning prototypes. Individual makers and small research teams are now able to develop complex systems that were previously accessible only to industrial or academic institutions.

A key factor that enabled this dynamic development was the emergence of open-source technologies. The internet allowed inventors from all around the world to share their concepts and collaborate with each other, creating free-to-use software frameworks, design tools, and complete hardware projects with publicly available documentation and source files.

Robotics is an area in which these trends are clearly visible. The combination of mechanical design, electronics, and software makes robotic systems ideal for open-source development and rapid prototyping. As a result, robotic projects have become increasingly popular, with one of the biggest open-source frameworks created for robotics: ROS (Robot Operating System). It provides a collection of tools and libraries for robot development.

Despite this progress, robotic manipulators remain relatively inaccessible to individual users. Commercially available robot arms, such as compact desktop manipulators intended for educational or light industrial use, have a relatively high cost. Additionally, such systems often have hardware that cannot be modified or a closed software ecosystem, which makes them difficult to integrate into custom applications. Figure 1.1 shows the MechArm 270 [10], a commercially available compact robotic arm designed primarily for educational purposes. It has a small size and weight, which can help in learning to operate the robot while it is situated on a desk, but its high price makes it inaccessible for most individual makers.



Figure 1.1. MechArm 6-DOF robot from Elephant Robotics

On the other hand, existing open-source robotic arm projects provide detailed documentation and freely available designs, but they typically have a high material costs, long build times, and a high level of technical complexity. Figure 1.2 shows the AR4 [5], a popular open-source robot arm, which can be used in serious tasks such as welding or removing parts from a CNC machine, as proven by the user community. The drawback of this robot, besides the high cost of materials, is the complexity of the build. Creating it requires a broad set of technical skills, which makes it inaccessible for beginners and poses a challenge even for intermediate makers.



Figure 1.2. The AR4 open-source 6-DOF robotic arm

Figure 1.3 shows another popular open-source robotic arm, the BCN3D Moveo [6]. It relies heavily on 3D printing for its mechanical components and increases the torque of the stepper motors through the use of belt drives. Similar to the AR4, this robot has a high cost of materials and is difficult to manufacture and assemble.



Figure 1.3. BCN3D Moveo open-source robotic arm

This thesis presents the design process of a low-cost, six-degree-of-freedom robotic arm intended for simple assembly and manipulation tasks that showcase the implementation of open-source technologies. The work addresses the gap between commercially available, closed robotic systems and existing open-source solutions that, while flexible and well documented, often remain costly and complex to implement.

1.1. Main goals of the project

The primary objective of the project was to create an easy-to-use robotic arm suitable for basic assembly tasks, such as soldering or component handling. The project was developed under the following constraints and assumptions:

1. Design and construct a fully functional robotic arm using open-source tools and technologies.
2. Demonstrate creative use of 3D printing technology for prototyping custom mechanical components.
3. Integrate the robotic system within the ROS 2 framework to ensure modularity and extensibility.
4. Utilize ROS 2 tools for visualization and motion planning.

1.2. Structure of the thesis

Chapter 2 introduces the theoretical background and key technologies used in the project. The mechanical aspects of the robotic arm, including the design process, assembly, and final construction, are presented in Chapter 3. Chapter 4 focuses on the electronic hardware of the system and its integration with the software layer. The software architecture and the implementation of ROS 2-based tools are described in Chapter 5. Finally, Chapter 6 summarizes the results obtained in this thesis and outlines potential directions for future improvements.

2. Theoretical background

This chapter aims to provide the basic theoretical background for the tools used during the development of the robot arm. It introduces the core concepts of ROS 2 programming model that are essential for understanding the software architecture of the robotic system. Additionally, it presents key technologies and specialized tools used during the development of the robot.

2.1. ROS 2 framework

ROS 2 [3] - Robot Operating System 2 is an open-source framework. It was designed to simplify the development of complex robotic systems by introducing modularity and dividing the project into a collection of independent components that communicate through standardized interfaces. This approach made the software easy to reuse and allows for rapid prototyping. In addition, ROS 2 provides a rich ecosystem of tools that support visualization, simulation, motion planning, and system integration.

2.1.1. ROS 1 vs ROS 2

ROS 1 was first released in 2007 and quickly became a standard in robotics. Supported and constantly developed by communities around the world, it grew and evolved to finally become its successor: ROS 2. While ROS 2 is a continuation of the ROS 1 project, it introduced changes at a fundamental level after the limitations of ROS 1 became apparent, particularly in real-time performance, reliability, and multi-platform support. Instead of updating ROS 1, ROS 2 was developed as a new framework that addressed these problems.

As described in the article published on the ROS 2 Design documentation [16], ROS 2 introduced architectural changes that significantly improved system robustness and flexibility. One of the most important differences is the use of a Data Distribution Service (DDS) middleware for communication, which enables decentralized, scalable, and configurable data exchange. Furthermore, while ROS 1 worked mainly with C++03 and targeted Python 2, ROS 2 adopted modern programming standards, including C++11 and Python version 3. Finally, it provides improved support for real-time systems and embedded platforms.

Due to these improvements, ROS 2 was selected as the software framework for this project. However, because of architectural differences between ROS 1 and ROS 2, not all existing packages and libraries

are directly compatible. As a result, careful selection of tools and versions is required when developing ROS 2-based systems.

2.2. Basics of ROS 2

The fundamental concepts of the ROS 2 programming model are nodes and interfaces that provides communication between them.

2.2.1. Nodes

ROS 2 systems consist of independent processes called nodes. Each node should be responsible for a specific task within the system. It can act as a publisher, a subscriber, or both. As an example, a publisher node responsible for data acquisition from a sensor can send messages, and a subscriber node can receive the data and use it for its own process.

The primary advantage of this approach is modularity. Nodes are independent, reusable and easy to use in different configuration. This allows for experimentation and scalability of ROS 2 projects. A single executable can consist of one or more nodes that work simultaneously. Nodes can be implemented in C++ or Python, and allow the use of external libraries not directly related to ROS 2.

2.2.2. Communication interfaces

Nodes communicate with each other using standardized communication interfaces, including topics, services, and actions.

Topics are the most common form of communication between nodes. They enable asynchronous data exchange in a publisher-subscriber model and are used for continuous data streams, such as joint states or sensor measurements.

Services contrary to topics do not work continuously but provide a synchronous call-response mechanism and are suitable for short, discrete operations. Actions extend the service concept by supporting long-running tasks that provide feedback and can be preempted, making them particularly useful for motion execution and trajectory control.

Data exchanged between nodes is done by using messages. Message types can be defined for a specific purpose or selected from standardized message definitions provided by ROS 2 packages [17].

2.3. URDF and SRDF

The Unified Robot Description Format (URDF) [23] is an XML-based format used by ROS 2 tools to describe the physical structure of a robot. A URDF defines the robot as a collection of rigid links connected by joints, specifying their spatial relationships and kinematic structure. Using the XML specification [24], a URDF file can describe joint types, joint limits, link geometries, collision models, and inertial properties. URDF files are used in ROS 2 tools for visualization and simulation.

The Semantic Robot Description File (SRDF) [21] is derived from URDF. It adds the semantic information required for advanced motion planning by organizing links into planning groups, defining end-effectors, and creating predefined poses. Additionally, it disables collision pairs to avoid accidental collision detection. It is essential for motion planning frameworks, as it defines how the robot should be interpreted and controlled, in contrast to URDF, which describes physical construction.

2.4. Large community projects: RViz, MoveIt 2, Micro-ROS

One of the main reasons why ROS 2 was chosen for the project is the availability of advanced tools that support programming, debugging and testing a robotic system. These tools are developed by various organizations [18] and maintained by contributors all over the globe,

2.4.1. RViz

RViz [19] is a package that visualizes the URDF of a robot in three-dimensional environment. It can be used to easily inspect and debug the robot joint transforms, link geometry, and collision boxes. The RViz environment is interactive, allowing the joints to be tested, which makes it particularly useful during the development and validation of the robot description.

In this project, RViz was used to verify the robot's kinematic structure, joint orientations, and motion limits, as well as to visualize trajectories generated by motion planning algorithms.

2.4.2. MoveIt 2

MoveIt 2 [13] (referred throughout this work as MoveIt) is a framework for motion planning, manipulation, and kinematics. It provides tools for solving inverse kinematics, planning collision-free trajectories, and executing motions on robotic manipulators based on the SRDF file of a robot. When combined with RViz, MoveIt enables interactive testing of tasks such as pick-and-place operations while considering obstacles defined in the planning scene.

MoveIt can be controlled either through its graphical interface or via C++ and Python APIs, allowing seamless integration into robotic applications. It significantly simplifies motion planning by abstracting complex kinematic computations and providing reliable planning algorithms, including support for redundant manipulators. Figure 2.1 shows the model of the robot visualized in RViz with the MoveIt interface. The green robot model represents the starting position, and the orange model represents the goal position. MoveIt allows the user to grab the end effector represented by a sphere and place it in the desired orientation and position.

Additionally, MoveIt includes a Setup Assistant tool that allows for the generation of a functional SRDF from a URDF model. In this project, MoveIt was used to generate the SRDF, compute inverse kinematics solutions, and plan motion trajectories for the robotic arm.

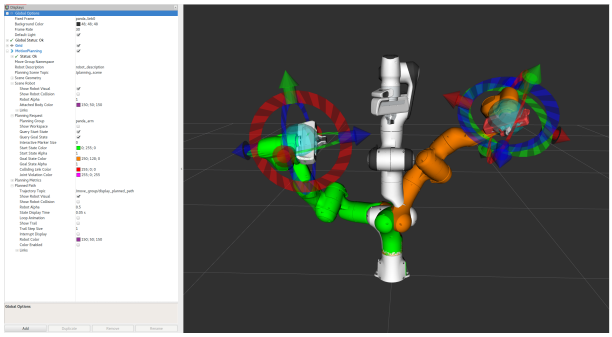


Figure 2.1. Franka Emika Panda robot in RViz operated using MoveIt

2.4.3. Micro-ROS

Micro-ROS [11] extends the ROS 2 ecosystem to microcontrollers by making it act as a node. Communication with the rest of the system is achieved through a serial or wireless connection using standard ROS 2 interfaces. Figure 2.2 represents the architecture of a system that uses Micro-ROS.

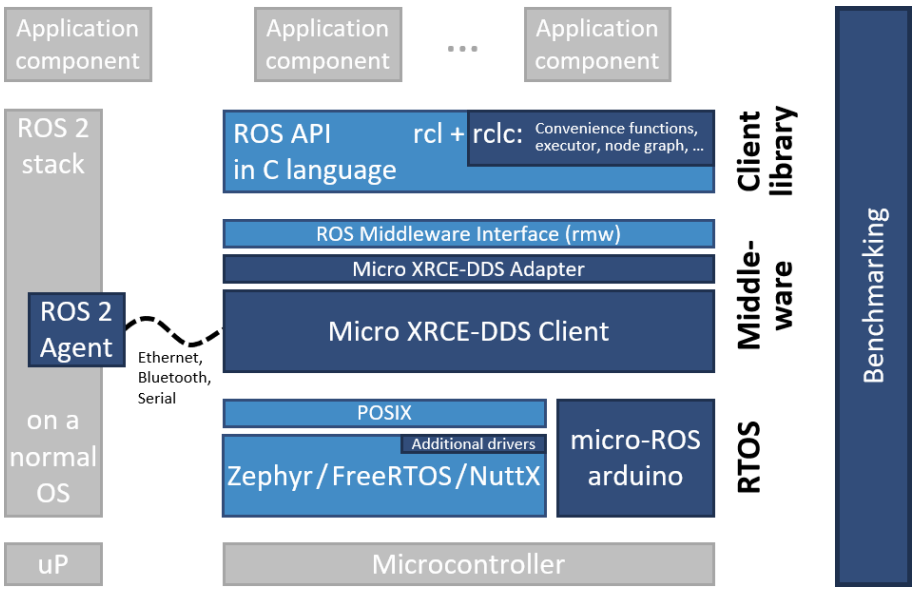


Figure 2.2. Micro-ROS architecture [11]

This approach allows low-level hardware control, such as controlling servomotors and sensor data acquisition, to be implemented on microcontrollers while higher-level planning and control algorithms are running on a host computer. In the presented system, Micro-ROS provides a bridge between the robotic arm's embedded electronics and the ROS 2-based control architecture.

3. Mechanical design of the robot

Mechanical design is a crucial part of any robotic system. It must account for the forces acting on the structure and be designed within the geometrical constraints. The design should balance performance with the cost of materials and the difficulty of manufacturing custom parts. An effective mechanical design also allows for easy assembly, maintenance, and future modifications, which is particularly important in prototype and research-oriented robotic systems.

3.1. Kinematic structure and degree-of-freedom selection

One of the first steps in the mechanical design process is selecting an appropriate kinematic structure for the intended application. That involves analyzing workspace of the robot, types of operations it should be capable of performing, and the relative importance of factors such as speed, accuracy, and flexibility.

The goal of this project is to develop a highly versatile robotic arm that can be adapted to a wide range of tasks rather than being optimized for a single application. This requirement strongly influences both the kinematic structure and the number of degrees of freedom of the robot.

3.1.1. Degrees of freedom

Robot movements are constrained by its number of degrees of freedom (DOF). A robot arm with four degrees of freedom provides limited movability and is restricted to simple tasks where the position of the tool center point (TCP) needs to be controlled, while the orientation is fixed. In contrast, a six-degree-of-freedom manipulator allows for full control of both position and orientation of the TCP in three-dimensional space.

Increasing the number of degrees of freedom beyond six introduces kinematic redundancy. While it gives the advantage in situations where the arm is required to avoid obstacles, it also increases mechanical complexity and computation of inverse kinematics is more demanding and requires the use of numerical methods.

For the purposes of this project, redundancy is not a strict requirement. A six-degree-of-freedom configuration provides sufficient dexterity and full control of the TCP while maintaining a reasonable balance between mechanical complexity, control effort, and versatility. Therefore, a 6-DOF architecture was selected as the most suitable solution.

3.1.2. Final selected layout

The final mechanical layout of the robot is based on an anthropomorphic structure, consisting of a shoulder, elbow, and wrist assembly. The orientation of the joint axes and the placement of actuators were chosen to achieve a balance between mechanical complexity, achievable dexterity, and ease of manufacturing.

To illustrate the adopted kinematic concept, a simplified schematic representation of the robot arm is shown in Figure 3.1. The diagrams are not intended to reflect exact dimensions but rather to present the serial kinematic chain, joint ordering, and link relationships used in the design.

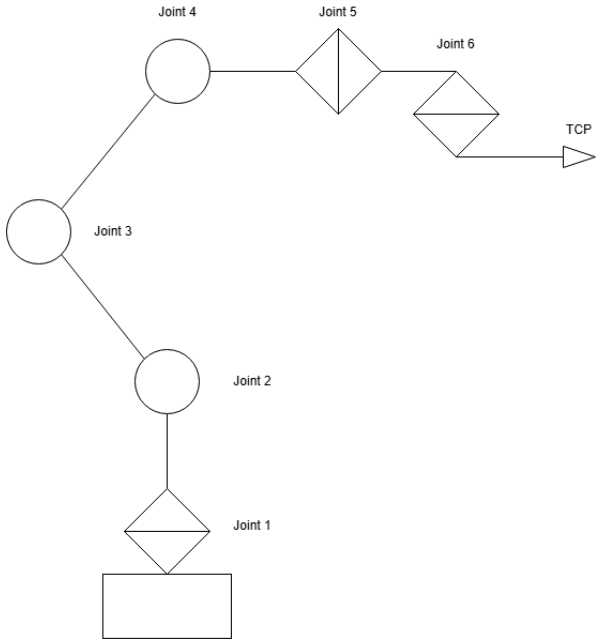


Figure 3.1. Joint Layout

3.2. Actuator selection and torque calculations

The selection of the joint motors was based on availability, price, and difficulty to control.

The most critical joint from a torque perspective is the second joint (shoulder pitch). It needs to provide enough torque to lift the whole arm when it is in an extended position and have enough torque left to lift the held object. Even when the mass of the moved object is small, the length of the lever arm significantly increases the required torque. Therefore, the second arm motor used in the project was a high-torque Feetech FI7635M capable of generating up to 35 kg·cm when powered at 7.4 V.

For joints 3 and 4, the motors of choice were servomotors Feetech FT5325M, which operate on the same voltage as the servomotor chosen for the first joint but generate 25 kg·cm of torque. They were chosen because of their lower cost.

The first joint, responsible for base rotation, is actuated using a JK42HS48-1204 stepper motor. In this case, the torque requirements are related to the rotational inertia of the structure and the desired

angular acceleration rather than gravitational loading. As a result, torque requirements for this joint are less restrictive compared to the shoulder and elbow joints. In addition, the axis is connected to the stepper motor shaft using a 2GT belt and gears. If insufficient torque is observed during testing, the effective output torque can be increased by changing the gear-to-gear ratio by increasing the number of teeth on the joint shaft gear, or by reducing the angular acceleration of the first axis.

The wrist joints (joints 5 and 6) are subjected to relatively low loads, as they only support the end effector and do not contribute significantly to lifting the arm. Therefore, micro servomotors Tower Pro MG-90S were used for these joints, as their torque capabilities of 2.2 kg·cm are sufficient for the expected operating conditions.

The torque estimations were based on a simplified static model assuming worst-case loading conditions in which the robotic arm is fully extended in the horizontal plane. This configuration maximizes gravitational torque and therefore represents the most demanding static operating scenario for the actuators.

Dynamic effects such as acceleration, deceleration, and inertial forces were neglected in the analytical model. Instead of explicitly modeling these effects, they were compensated for by selecting actuators with a significant torque safety margin.

To further increase robustness, a conservative mass distribution was assumed. The combined mass of each printed link and its associated servomotor was approximated as being concentrated at the corresponding joint axis. While this assumption does not reflect the true mass distribution of the system, it intentionally overestimates the resulting torque and ensures a pessimistic torque requirement.

All mass values were rounded up to account for uncertainties in material density, fasteners, wiring, and manufacturing tolerances. As a result, the calculated torque values represent lower bounds for actuator selection rather than precise operational torques.

The mass of the servomotors combined with the mass of the arm was located in the joint position to add to the margin. The results are presented in Table 3.1

Table 3.1. Conservative estimates of minimum static torque requirements

Joint	**Minimum torque required [kg·cm]**
Joint 2 (Shoulder)	7.5
Joint 3 (Elbow)	3.5
Joint 4 (Wrist pitch)	1

The selected actuators provide a sufficient safety margin over the estimated minimum torque requirements, even in the worst-case configuration when the arm is fully extended. This margin compensates for modeling inaccuracies, dynamic effects, and additional loads.

3.3. Material selection and manufacturing methods

3D printing technology has greatly improved in recent years and became an essential skill for modern engineers. It allows for prototyping parts in a short amount of time with good accuracy, and the cost of manufacturing parts is very low. This makes it perfect for the iterative design process of robotic systems.

3.3.1. Manufacturing approach

The most common method of 3D printing is Fused Deposition Modeling (FDM). This method is easy to work with, low-cost, fast, and allows for creating geometries impossible to manufacture using different methods such as CNC machining. For the first iteration of the robot all mechanical parts were manufactured using FDM technology. One of the parts printed for the robot was a GT2 pulley, if the accuracy of the FDM turned out to be insufficient, the pulley could be reprinted using Stereolithography (SLA). It uses resin as the print material, and can achieve layer heights as small as 0.025 mm, resulting in improved surface finish and dimensional precision.

3.3.2. Material comparison

The filament used in FDM can be manufactured to have specific mechanical properties. It is crucial to choose the right filament type suitable for the project. Table 3.2 presents a comparison of commonly used FDM materials based on data summarized in an ALL3DP article [4]. Figure 3.2 visualizes the advantages and disadvantages of the most popular filaments.

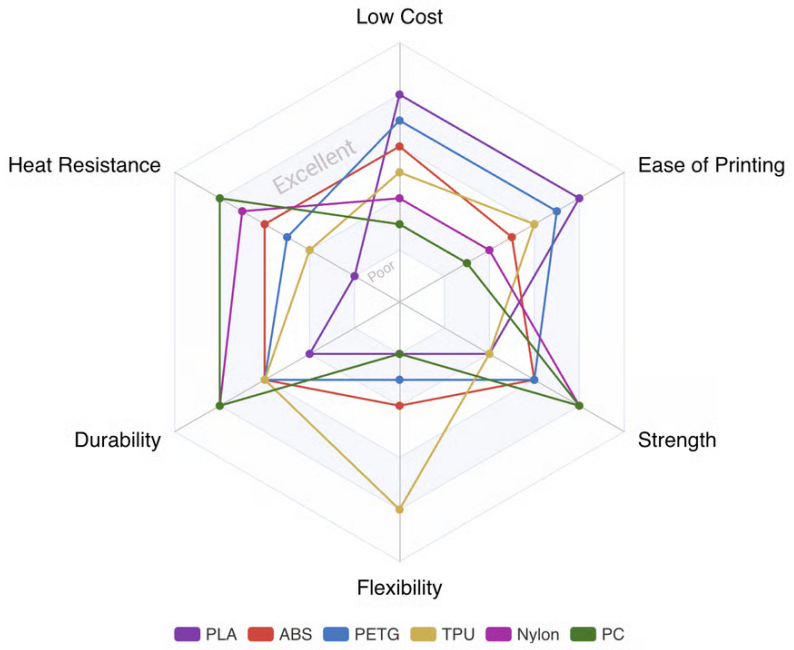


Figure 3.2. Filament types presented by their advantages and disadvantages [4]

Table 3.2. Comparison of selected FDM filaments

Material	Advantages	Disadvantages
PLA	Very easy to print, and thus great for prototyping; low warping; good dimensional accuracy; low cost.	Low temperature resistance; relatively low impact strength; not suitable for functional, load-bearing parts.
ABS	Higher mechanical strength than PLA; good impact resistance; moderate thermal resistance.	Difficult to print (warping, necessary to print in chamber because of fumes); lower dimensional accuracy than PLA.
PETG	Good balance of strength and printability; higher temperature resistance than PLA; minimal warping.	Slightly less rigid than PLA (problematic in robotics); lower impact resistance than ABS.
TPU	Flexible and elastic; great impact absorption; highly durable.	Difficult to print; not suitable for rigid structural components.
Nylon (PA)	Very strong and wear-resistant; excellent impact toughness; good thermal resistance.	High moisture absorption; warping risk; requires high printing temperature.
PC (Polycarbonate)	Extremely strong and heat-resistant; suitable for heavy-duty functional parts.	Very difficult to print - very high printing temperature and requires enclosure; demanding to tune and prone to warping.

For the prototyping phase, PLA was the filament of choice because of its ease of printing, low cost, and great dimensional accuracy. This allowed focusing on robot geometry and adjustment of tolerances. In future iterations, the material could be changed to a more durable one, such as PETG or ABS.

3.3.3. Inserts, tolerances, and mechanical joints

Although FDM 3D printing offers good dimensional accuracy, parts must be designed specifically with the limitations of the process in mind. During the design phase, large overhangs were avoided to reduce the need for support structures and to improve surface quality. For components requiring press-fit assembly, a dimensional offset of approximately 0.1 mm was applied. Achieving reliable fits required several iterations of test prints and gradual adjustment of tolerances.

To ensure smooth rotation in joints, the parts are connected with ball bearings. This significantly reduced friction and wear compared to direct plastic-on-plastic contact. The bearings were mounted in the 3D-printed parts using a press-fit assembly.

Rigid connections between 3D-printed parts can be achieved using press fits, adhesive bonding, or threaded fasteners. However, press fits alone limit serviceability, while self-tapping screws can damage the printed material over time. To ensure that the project can be assembled and disassembled without

damaging the structure, brass threaded inserts, shown in Figure 3.3, were used. Inserts were heat-set into the printed components using a soldering iron, creating durable and reliable threaded connections.

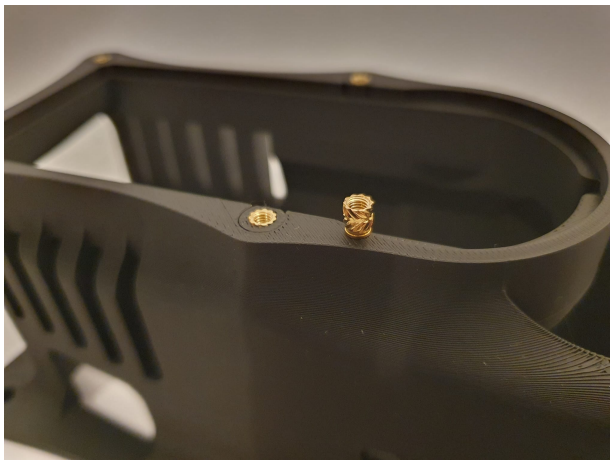


Figure 3.3. Brass threaded inserts

To ensure proper retention of the inserts, the diameter of the printed holes was designed to be slightly smaller than the nominal insert diameter, allowing the surrounding plastic to melt and secure the insert during installation.

3.3.4. Slicer settings and print orientation

An important aspect of FDM manufacturing is the orientation of parts on the print bed, as it directly determines the layer direction and therefore influences both mechanical strength and dimensional accuracy. Aligning layers with the expected load paths was a key consideration during slicing.

Print orientation also affects the accuracy of cylindrical features. Holes printed with their axes aligned vertically typically exhibit higher roundness and dimensional precision compared to holes printed horizontally. For this reason, components containing bearing seats were oriented such that these features were printed from the top, even when this required additional support material, which is shown in Figure 3.4.

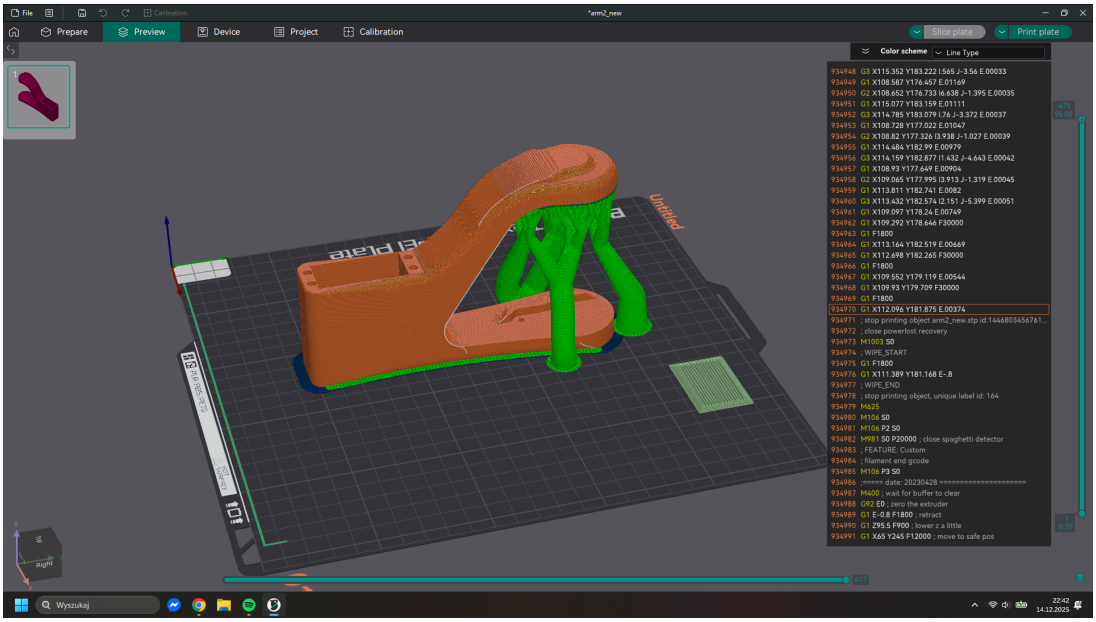


Figure 3.4. Positioning a part in slicer. The generated support structure is highlighted in green. The supports are added to ensure proper printing of overhanging features and to maintain geometric stability during the manufacturing process

For components containing heat-set brass inserts, additional material is required around the inserts to prevent weakening of the structure. This was achieved by applying local modifiers in the slicer software to increase wall thickness around the insert holes. Figure 3.5 shows the use of a local cylindrical modifier that is set to a higher number of walls.

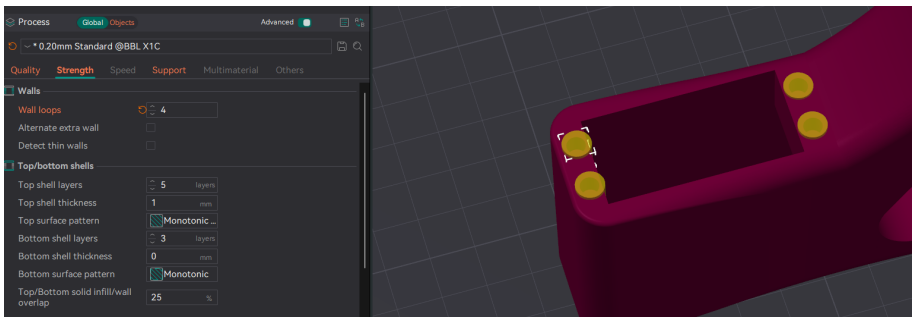


Figure 3.5. Local modifiers in the slicer

Infill density was selected based on the functional role of each part. Structural components were printed with higher infill percentages to improve stiffness, while non-load-bearing parts used lower infill to reduce material usage and print time. The infill pattern used throughout the project was gyroid, as it grants strength from all directions.

3.4. Base (first joint) mechanical construction

The design of the robot base posed one of the most significant mechanical challenges of the project. The base serves multiple functions: it houses the electronic components, provides adequate airflow for cooling them, ensures structural rigidity, allows convenient access for maintenance, and simultaneously acts as the first rotational joint of the robotic arm.

3.4.1. Structural design

To ensure ease of assembly and maintenance, the base was designed as a two-part structure connected using bolts and heat-set brass inserts. This modular approach allows the base to be opened repeatedly without damaging the printed material and enables easy use of the robot arm in a different configuration.

The lower part of the base serves as a structural foundation and can be rigidly mounted to a wooden or metal support plate. This improves overall stability and reduces vibrations during operation. The upper part of the base integrates the rotating elements of the first joint and supports the arm structure.

3.4.2. Electronics integration

The lower section of the base contains a dedicated electronics mounting platform, conceptually similar to a computer motherboard tray. The custom control board and DC-DC buck converter are mounted using brass inserts to ensure mechanical stability and electrical insulation.

Electronic components such as the stepper motor driver and power converters generate heat during operation, which can negatively affect performance and reliability. To avoid this, forced airflow was implemented using a 12 V DC fan mounted at the front of the base. Ventilation openings at the rear allow air to flow through the entire enclosure, effectively reducing the temperature inside.

3.4.3. First axis rotation mechanism

The first rotational axis connects the robotic arm to the base and must provide high rigidity while allowing smooth, low-friction motion. To achieve this, the joint is supported by two radial ball bearings, which ensure precise alignment and resistance to bending moments. Additionally, a needle bearing is used to carry axial loads resulting from the weight of the arm, preventing increased friction under vertical loading.

The first axis is driven by a stepper motor via a GT2 timing belt transmission. This solution allows for flexible torque adjustment through pulley ratio selection and isolates the motor from axial loads. Additionally, such design allowed for the implementation of a hollow rotating shaft. This enables the routing of power and signal cables through the joint using a slip ring, eliminating cable twisting and allowing unlimited rotation of the base. The idea is visualized in the cross section of the base in Figure 3.6. The finished first axis is shown in Figure 3.7. Both parts of the base, when assembled, are shown in Figure 3.8.

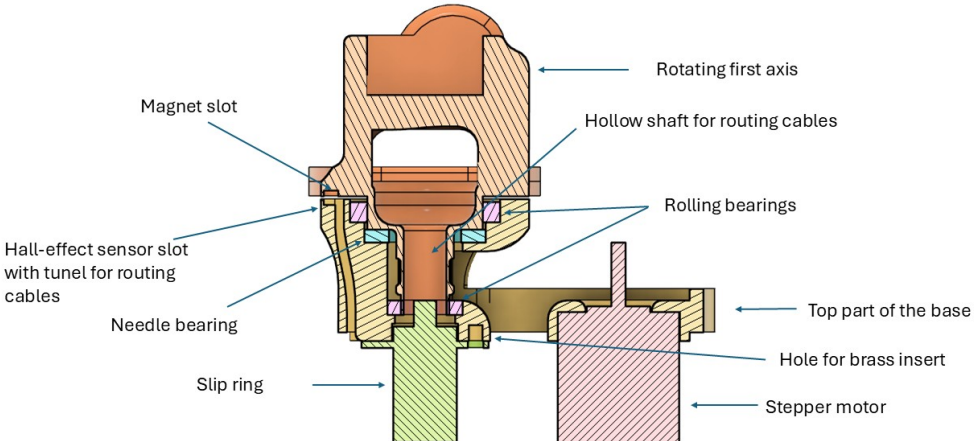


Figure 3.6. Top part of the base cross section

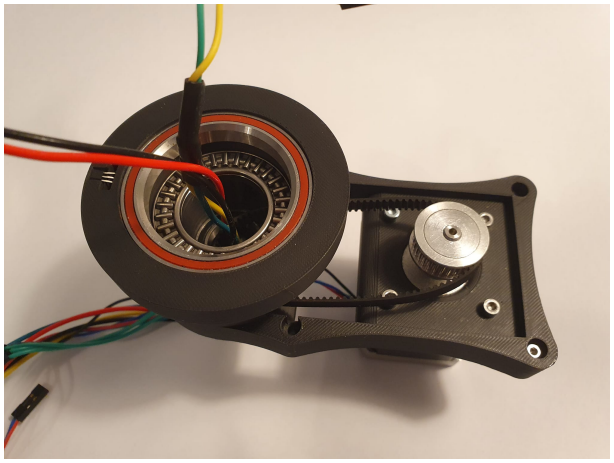


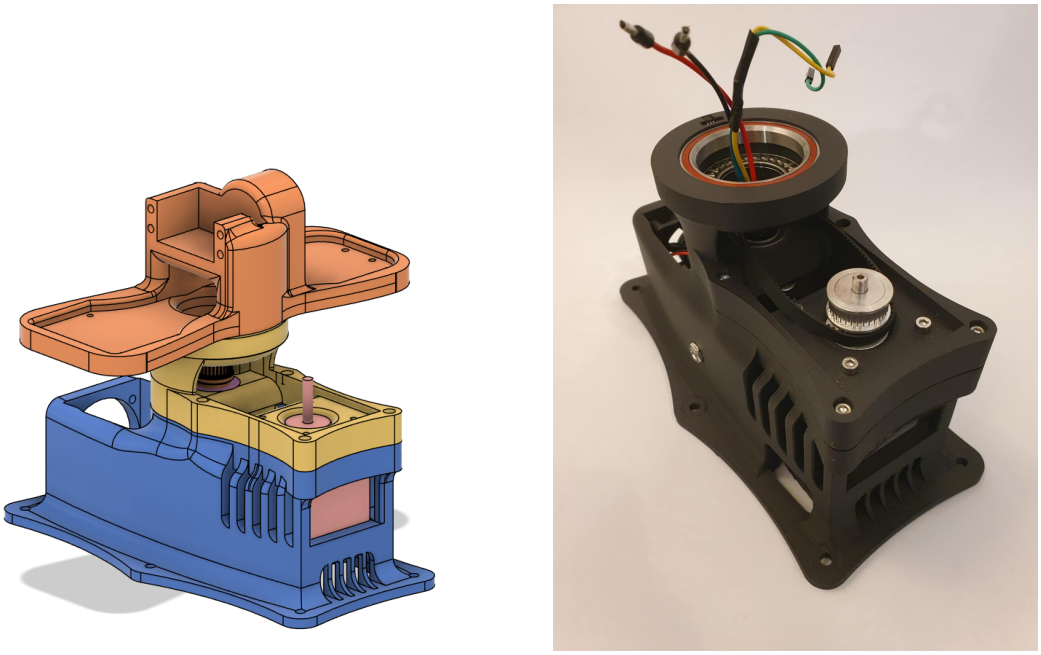
Figure 3.7. Assembly of the first axis

3.5. Arm structure

Using stepper motors in joints requires the implementation of an external gear system to increase torque. This increases mechanical complexity, mass, and manufacturing effort, particularly when custom gears must be fabricated. To avoid this and reduce the mass of the arm, joints two through six were designed around standard-sized servomotors. These actuators incorporate an internal gear system that allows them to deliver high torque while remaining relatively small and lightweight. This approach simplifies the mechanical design and reduces the number of custom components.

3.5.1. Shoulder and elbow joint (second and third axis)

The shoulder and elbow joints are subjected to the highest mechanical loads in the robotic arm, as they must support the mass of all other links, actuators, and the payload. For this reason, these joints were designed with a strong emphasis on rigidity and precise alignment. In the project, both links are exactly the same but rotated 180 degrees. High rigidity was achieved by designing tight-tolerance servomotor



(a) (b)

Figure 3.8. Assembled Base: (**a**) CAD visualization, (**b**) real photo

mounting and securing the motors using bolts and heat-set brass inserts. The smooth rotation is achieved by using a ball bearing, and connections are designed to snap into place.

3.5.2. Wrist (axes 4-6)

The wrist assembly is responsible for controlling the orientation of the end effector and must provide high dexterity while maintaining low inertia. Unlike the shoulder and elbow joints, the wrist joints are subjected to relatively low mechanical loads. Axis 4 does contribute to lifting the arm, but axes 5 and 6 primarily support the end effector.

For this reason, axis 4 was controlled by the same servomotor as axis 3, and compact servomotors with lower torque ratings were selected for axes five and six. The wrist was designed as a modular assembly, allowing individual joints to be accessed and serviced independently.

3.5.3. Cable management strategy

To protect power and signal cables and improve the overall appearance and reliability of the system, all wiring was routed internally through the arm structure. The arm components take advantage of 3D printing technology and were designed with internal channels running along their entire length, allowing cables to pass between joints without external exposure. Manufacturing similar parts using subtractive technologies would be significantly more difficult. At the joint interfaces, cables are routed through the inner diameter of the ball bearings. This solution prevents cable bending and abrasion during joint rotation while maintaining a compact and mechanically clean design. Figure 3.9 shows the cross section

of the assembled arm in CAD, with visible cable routing. Figure 3.10 shows how cables are led through the bearing during the assembly of the robot.

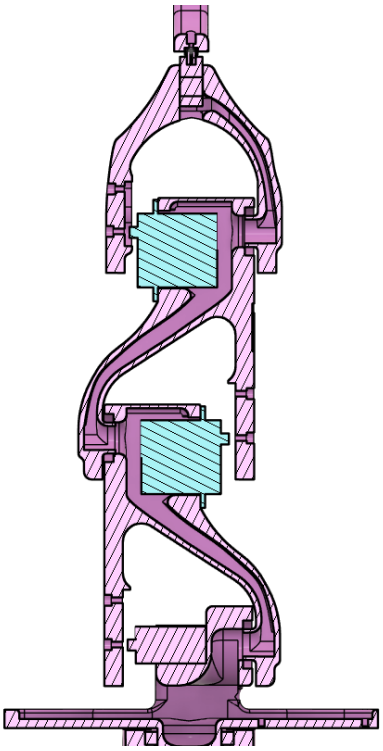


Figure 3.9. Cable routing

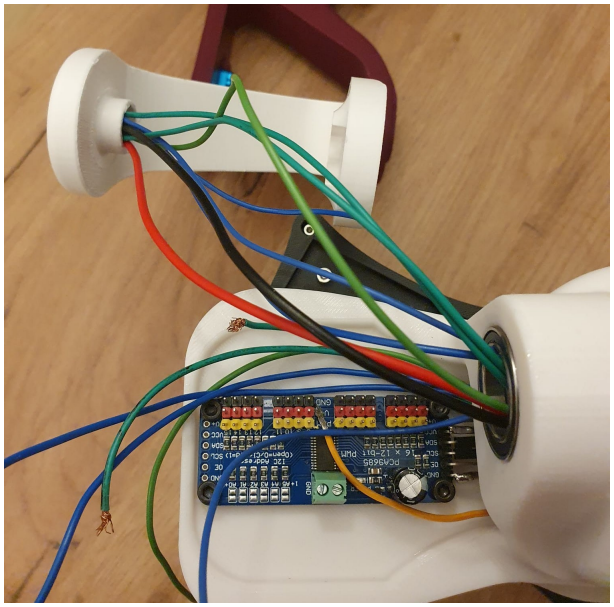


Figure 3.10. Routing of cables through the joint bearing

3.6. End effector

The end effector was designed to be lightweight and modular, allowing easy attachment of various tools depending on the application. Low-power micro servomotors were used to actuate the final joints, as the required torques are minimal and limited to tool manipulation.

The mechanical interface of the end effector enables quick replacement or modification, making the system suitable for experimentation and future extensions.

3.6.1. TCP reference tool

During initial testing and calibration, a simple reference tool was designed to visualize the TCP. The tool represents the local coordinate axes x, y, and z, allowing intuitive verification of position and orientation in both the physical system and the ROS 2 visualization environment. Figure 3.11 shows the finished TCP tool. Filaments with different colors were used to represent three axes.

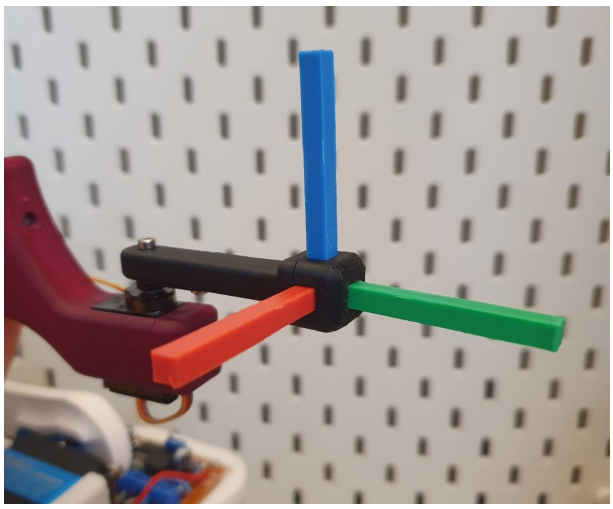


Figure 3.11. Tool representing TCP

This reference tool proved useful for validating kinematic models and ensuring consistency between the mechanical design and software representation.

3.7. Assembly, tolerances, and iteration process

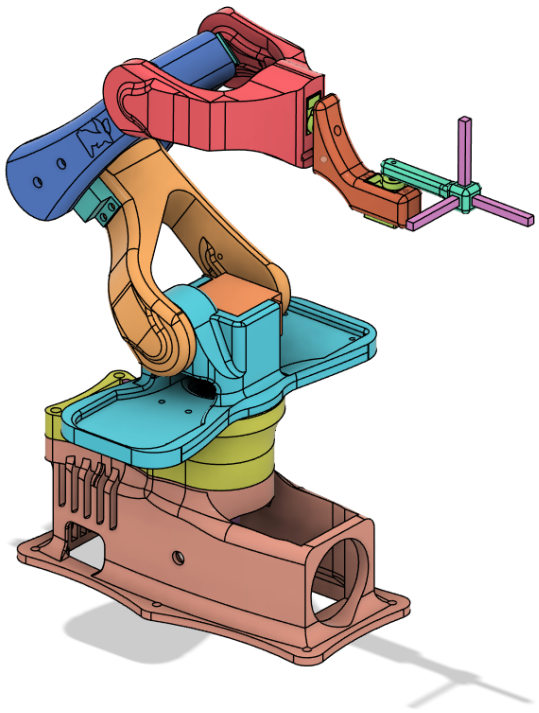
The mechanical design of the robot underwent several iterations during development. Each iteration focused on improving joint stiffness, fit accuracy, and ease of assembly. Test prints were used to validate tolerances, bearing fits, and insert placement before finalizing the designs.

This iterative approach allowed design issues to be identified early and resolved efficiently, resulting in a mechanically robust and serviceable final structure.

3.8. Summary of mechanical design

The mechanical design of the robotic arm was developed with a strong emphasis on modularity, manufacturability, and cost efficiency. The use of FDM 3D printing enabled rapid prototyping and iterative refinement, as well as the creation of internal passages for cable routing. The use of servomotors simplified actuator integration.

The final design achieves a balance between structural rigidity and low mass, supports reliable internal cable routing, and provides sufficient dexterity for a wide range of manipulation tasks. To illustrate the completed mechanical system, Figure 3.12 shows the assembled robot in CAD software and the finished assembled robot.



(a) Full robot assembly in CAD



(b) Finished assembled robot

Figure 3.12. Comparison of the robot design in CAD and the final physical assembly

4. Electrical design of the robot

4.1. System architecture overview

The robotic system was divided into two main functional subsystems: a low-level hardware control subsystem and a high-level control and computation subsystem. This separation allows time-critical tasks to be handled deterministically, while higher-level planning, visualization and user interaction are handled independently.

The high-level subsystem is implemented on a host computer running ROS 2 and is responsible for motion planning, trajectory generation, system supervision, and visualization. Low-level actuator control is delegated to a dedicated microcontroller operating in real time. Figure 4.1 presents a functional overview of the system architecture.

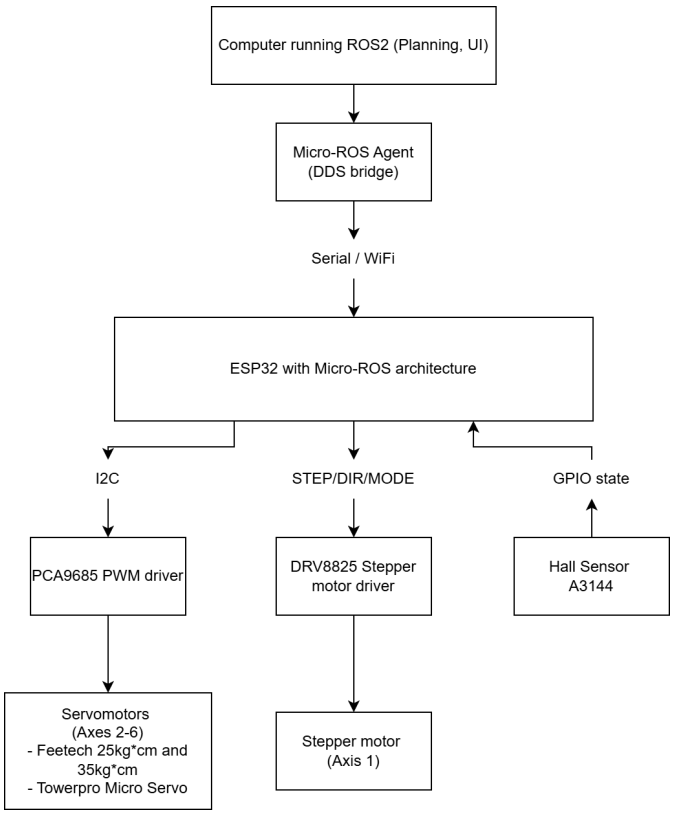


Figure 4.1. System architecture diagram

The control system is required to:

– generate PWM signals for five servomotors,

– control a stepper motor driving the base joint,

– acquire a digital homing signal for establishing a reference position.

All components are integrated within the ROS 2 ecosystem.

4.1.1. ESP32 microcontroller

For the low-level hardware control, the ESP32 microcontroller was selected because of its computational resources, memory capacity, and official support for Micro-ROS integration [12]. Additionally, it has a built in Wi-Fi module, allowing for wireless communication in future iterations.

The ESP32 provides flexible GPIO configuration, allowing it to interface with the PWM driver via the I^2C bus and generate step and direction signals for the stepper motor driver.

4.1.2. Micro-ROS integration

Micro-ROS enables the integration of a microcontroller into a ROS 2 project. Communication with the main CPU and the rest of the system is established via a serial connection, allowing the ESP32 to operate as a ROS 2 node with a subscriber for receiving joint position commands and a publisher for transmitting sensor data.

For the project, a micro_ros_arduino library was used to integrate the ESP32 into the ROS 2 ecosystem. The version of the tool needs to be chosen precisely for the version of ROS 2 for the project. In the project, the Arduino IDE was used to flash the board. For Micro-ROS to work properly, the version of the IDE also needs to be adjusted to match the ROS 2 version.

4.2. Actuator control

Actuator control was divided according to actuator type and timing requirements. The system includes:

– one stepper motor,

– three high-torque standard servomotors,

– two micro servomotors.

Each actuator type imposes different electrical constraints.

4.2.1. Stepper motor and DRV8825 driver

The first joint of the robot is driven by a JK42HS48-1204 stepper motor controlled using a DRV8825 driver. The driver supports microstepping and allows adjustment of the phase current to match the motor's electrical characteristics.

Precise step timing is required for reliable stepper motor operation. High-level operating systems do not guarantee real-time behavior, which can lead to irregular step generation and loss of motion accuracy. For this reason, stepper motor control was delegated to the ESP32 microcontroller, which provides deterministic timing.

A custom prototyping board was designed to ensure reliable wiring and proper power distribution for the stepper motor and driver.

4.2.2. Hall sensor for homing

The stepper motor requires a homing procedure at startup to establish a known reference position. This was achieved using a permanent magnet and a digital Hall-effect sensor (A3144). The sensor outputs a low signal when exposed to a magnetic field, allowing precise detection of the home position.

A startup procedure was set up, where the robot slowly rotates clockwise until it reaches a low signal from the hall sensor. This ensures a consistent reference frame for motion planning and prevents accumulated positioning errors.

4.2.3. Servomotors

Joints 2 through 6 are actuated using servomotors with integrated gearboxes and position control. Three categories of servomotors were used, selected according to torque requirements derived from the mechanical analysis in Chapter 3. Table 4.1 summarizes the parameters of servomotors used in each joint.

Table 4.1. Servomotors used in joints

Joint number	Servomotor Model	Voltage [V]	Max. torque [kg·cm]	Stall current [A]
Joint 2	Feetech FI7635M	7.4	35	3.5
Joint 3	Feetech FT5325M	7.4	25	3.5
Joint 4	Feetech FT5325M	7.4	25	3.5
Joint 5	Towerpro MG-90S	5	1.8	0.7
Joint 6	Towerpro MG-90S	5	1.8	0.7

The achievable torque depends on the supply voltage and current capability of the power system. Worst-case current consumption was considered to ensure reliable operation under maximum load conditions.

4.2.4. PCA9685 PWM driver

Generating stable and precise PWM signals for multiple servomotors is a time-critical task that is inefficient to implement directly on a high-level system. For this reason, a dedicated PWM driver module based on the PCA9685 controller was used.

The PCA9685 provides up to 16 independent PWM channels with 12-bit resolution and communicates with the ESP32 via the I^2C bus. This solution ensures consistent PWM timing while reducing the processing load on the microcontroller.

The PCA9685 module was placed in the rotating part of the robot base. This arrangement allows all servomotor signals to remain local to the rotating structure, while only four wires pass through the slip ring: two power lines and two I^2C communication lines.

4.3. Power distribution and voltage regulation

The robotic system contains components requiring multiple supply voltages, ranging from high-current motor supplies to low-power logic rails. To simplify wiring and improve reliability, a single primary power source was used, from which all required voltage levels are derived.

4.3.1. Primary power supply

A 12 V DC power supply was selected as the primary power source, as it directly supplies the stepper motor and DC fan and allows efficient step-down conversion to lower voltages using switching regulators.

4.3.2. Voltage rails and regulation

The following voltage rails are used in the system:

- 12 V: stepper motor, cooling fan.
- 7.4 V: high-torque servomotors (joints 2–4).
- 5 V: PCA9685, micro servomotors.
- 3.3 V: ESP32 logic (internal regulation)

Switching DC–DC buck converters (LM2576 and XL4016) were used to step down the primary voltage with high efficiency. Figure 4.2 shows the voltage distribution throughout the project.

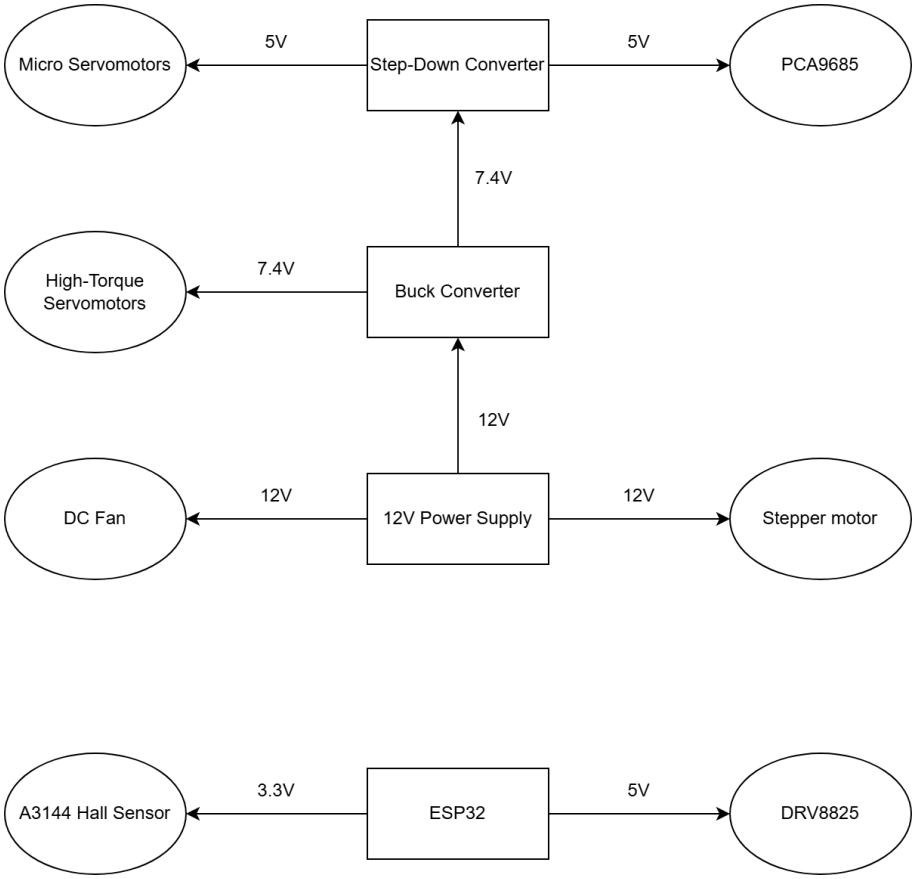


Figure 4.2. Voltage distribution

4.3.3. Current consumption estimation

Worst-case current consumption was estimated assuming a conservative operating scenario in which the servomotor driving the second joint operates near stall torque. Since subsequent joints have progressively shorter lever arms, the resulting torque — and therefore current draw — decreases along the kinematic chain. For DC servomotors, the drawn current was assumed to be proportional to the generated torque, which provides a reasonable approximation for worst-case analysis. Table 4.2 represents the approximated current drawn by the most important joints.

Table 4.2. Worst-case torque and current estimates for main servomotors

Joint	**Torque at joint [kg·cm]**	**Estimated current [A]**
Joint 2 (Shoulder)	35	3.5
Joint 3 (Elbow)	24	3.36
Joint 4 (Wrist pitch)	13.5	1.89

The two wrist joints are actuated using TowerPro micro servomotors, each with a stall current of approximately 0.7 A. In the worst-case scenario, the total current drawn by all servomotors is therefore approximately 10.15 A.

The base joint is driven by a stepper motor controlled using a DRV8825 driver. Unlike servomotors, the current drawn by a stepper motor is regulated by the driver and remains approximately constant regardless of load. For the selected motor, the maximum supply current drawn from the 12 V rail was estimated at approximately 1.5 A.

Combining both actuator groups yields a conservative worst-case current consumption of approximately 11.6 A. Since simultaneous stall conditions across all actuators are highly unlikely during normal operation, a 12 V / 10 A power supply was selected, providing a sufficient margin for safe and reliable operation.

The rotating upper part of the robot is electrically connected to the stationary base using a slip ring. This solution allows unlimited rotation of the base joint without twisting or damaging the power and communication cables.

The slip ring was selected with a continuous current rating of 10 A, which corresponds to the conservative worst-case current consumption estimated for the servomotor subsystem. Since the majority of high-current loads are located in the rotating structure, the slip ring primarily carries the combined supply current for the servomotors and control electronics.

In practice, the rated current of the slip ring is rarely reached during normal operation, as simultaneous stall conditions of all servomotors are highly unlikely. Nevertheless, selecting a slip ring with a current rating equal to the worst-case estimate provides an additional safety margin and improves system reliability.

4.4. Summary of electrical design

The final electrical system integrates the microcontroller, motor drivers, power converters, and peripheral modules on a custom-designed prototyping board, as shown in Figure 4.3. Buck converters were selected due to their high efficiency and ability to increase output current while reducing voltage, which is essential for high-current servo operation.

The final design ensures stable power delivery, deterministic actuator control, and seamless integration with the ROS 2-based software architecture.

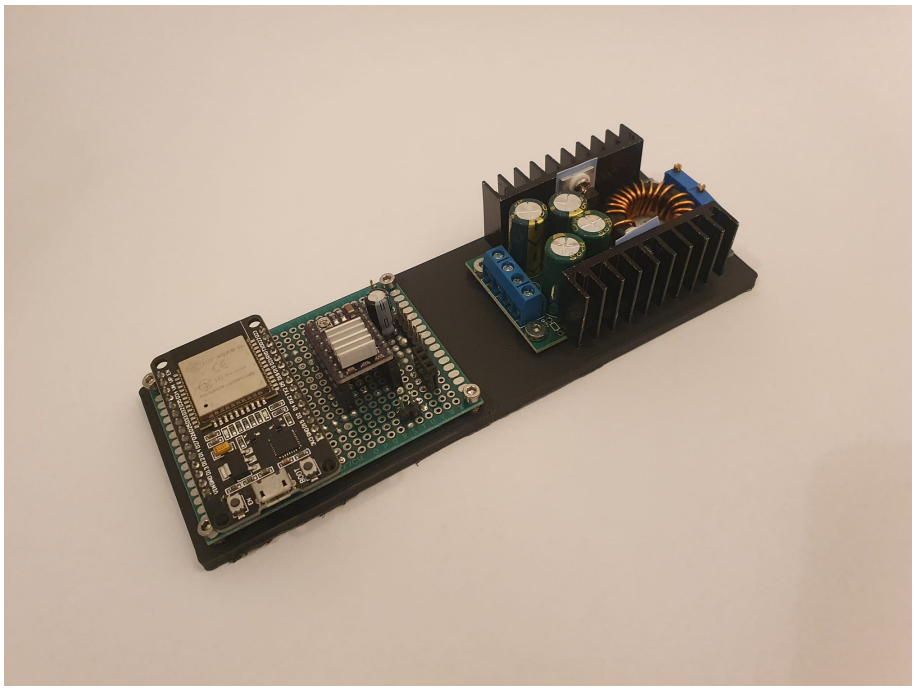


Figure 4.3. Board located in the base of the robot containing the custom prototyping board and buck converter

5. Programming the Robot with ROS 2

5.1. Software architecture overview

The software system follows a layered ROS 2-based architecture introduced in Chapter 4 and illustrated in Figure 4.1. High-level perception, motion planning, and user interaction tasks are executed on a PC running ROS 2, while time-critical motor control is delegated to an ESP32 microcontroller. Communication between both layers is achieved using the Micro-ROS framework.

5.2. ROS 2 environment

ROS 2 is an open-source framework that is actively developed, with multiple distributions available. Selecting an appropriate distribution at the beginning of a project is essential to ensure long-term support and compatibility with required tools.

Although ROS 2 supports multiple operating systems, it is primarily developed for Linux and is officially recommended for Ubuntu. Therefore, Ubuntu was selected as the operating system for this project.

At the time of development, Jazzy Jalisco was the newest ROS 2 distribution. However, the project uses ROS 2 Humble Hawksbill, which is a Long-Term Support (LTS) release. This version was chosen due to its proven stability and compatibility with MoveIt and Micro-ROS. ROS 2 Humble operates reliably on Ubuntu 22.04, which was installed on an external SSD drive.

After completing the installation according to the official documentation, the environment was verified using the `turtlesim` demo, which demonstrates core ROS 2 concepts such as nodes, topics, and message passing.

5.3. Robot description and visualization

To enable robot visualization and motion planning in ROS 2, a Unified Robot Description Format (URDF) file was created.

5.3.1. URDF creation

An URDF file can be created manually; however, the ROS community has developed tools that can create a URDF file directly from a CAD model [8]. Examples include the SolidWorks URDF exporter and the plugin to Blender. Since Autodesk software was used for mechanical design, the `fusion2urdf` [9] add-in was selected.

To ensure correct URDF generation, the CAD assembly was constructed according to the rules specified in the repository documentation. These rules include proper naming conventions, the use of lowercase link names without spaces, and defining the first link as `base_link`.

The add-in generates a ROS 1 package, which requires manual adaptation for ROS 2. This includes modifying the CMake configuration to remove `catkin` dependencies, creating a Python-based launch file, and adjusting the RViz configuration.

An important advantage of this approach is that the generated package includes STL meshes of the robot model, allowing direct visualization in ROS 2 tools. Additionally, inertia parameters are computed automatically, which is essential for motion planning and simulation. The inertial parameters shown in Listing 5.1 were automatically computed from the CAD model. Although inertial parameters are not strictly required for the semantic description itself, the MoveIt Setup Assistant requires a complete URDF model. Proper inertial definitions are therefore necessary to avoid configuration warnings and to ensure correct trajectory execution and simulation behavior.

```
<link name="arm3_1">
  <inertial>
    <origin xyz="0.029311091342528874 -0.00010331124504571292 0.04139134610850992"
    rpy="0 0 0"/>
    <mass value="0.1076032510643043"/>
    <inertia ixx="0.000128" iyy="0.000215" izz="0.000119" ixy="0.0" iyz="0.0" ixz="
    -1e-06"/>
  </inertial>
  <visual>
    <origin xyz="0.029 -0.00986 -0.39017" rpy="0 0 0"/>
    <geometry>
      <mesh filename="package://urdf5_description/meshes/arm3_1.stl" scale="0.001
    0.001 0.001"/>
    </geometry>
    <material name="silver"/>
  </visual>
  <collision>
    <origin xyz="0.029 -0.00986 -0.39017" rpy="0 0 0"/>
    <geometry>
      <mesh filename="package://urdf5_description/meshes/arm3_1.stl" scale="0.001
    0.001 0.001"/>
    </geometry>
  </collision>
</link>
```

Listing 5.1. Example description of link in the generated URDF

As shown in Listing 5.1, the visual and collision geometry are the same, based on the mesh of the robot. The link uses a material type that is described separately. In the project, a simple material with a gray color was created. The material definition used by the link is shown in Listing 5.2.

```
<material name="silver">
  <color rgba="0.700 0.700 0.700 1.000"/>
</material>
```

Listing 5.2. Material description

In the automatically generated URDF, the joint limits were initially set to the default range of $[-\pi, \pi]$, as shown in the Listing 5.3. However, joints 2 to 6 are driven by servomotors with a limited angular range of $180°$. Therefore, the limits were corrected to $[-\pi/2, \pi/2]$ to accurately reflect the mechanical constraints of the actuator.

```
<joint name="joint2" type="revolute">
  <limit lower="-3.14" upper="3.14" effort="10" velocity="1.0"/>
  <origin xyz="-0.029 0.00986 0.04025" rpy="0 0 0"/>
  <parent link="rotation_base_1"/>
  <child link="arm1_1"/>
  <axis xyz="1.0 0.0 0.0"/>
</joint>
```

Listing 5.3. Example description of a revolute joint in the generated URDF

The first joint was defined as a continuous joint, as it is actuated by a stepper motor without angular limits. This configuration allows for unrestricted rotation in the URDF model.

5.3.2. RViz visualization

To verify the correctness of the generated URDF, the robot model was visualized in RViz. Although RViz does not perform physical simulation, it is a valuable tool for inspecting robot geometry, joint configurations, coordinate frames, and collision models.

The robot model was visualized using a generated launch file that loads the URDF into RViz:

```
$ ros2 launch urdf_package display.launch.py
```

To verify the joint limits and motion directions, joint_state_ publisher tool was used. It publishes simulated joint states, allowing for manual manipulation of individual joints in RViz using sliders.

Figure 5.1 shows how RViz allows the user to visualize transforms of the robot and collision boxes. In this project, the collision boxes were identical to the geometry of the links, but they can be simplified in future iterations to improve computational efficiency or increased in size to ensure safety.

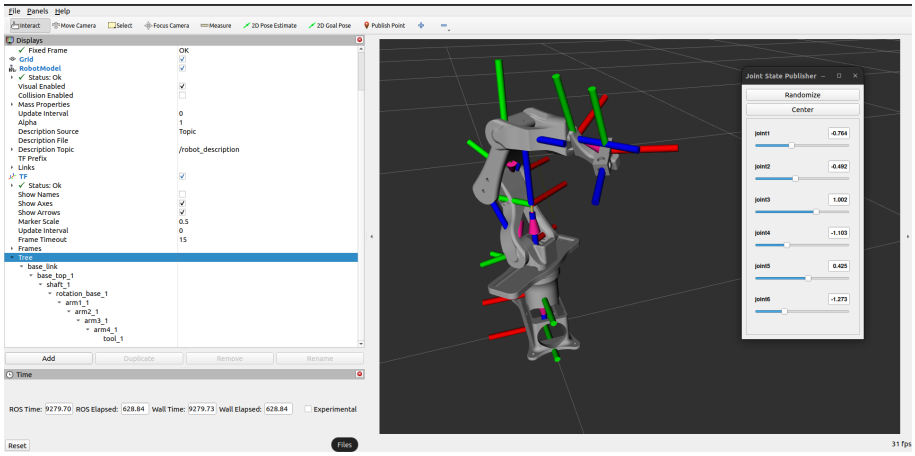


Figure 5.1. Visualization of robot coordinate frames and collision geometry in RViz, and joint_state_publisher sliders

5.4. Motion planning with MoveIt

After creating a URDF file of the robot, a Semantic Robot Description Format (SRDF) is required to define planning groups, assign an end effector, and generate a self-collision matrix. MoveIt has a special tool dedicated to creating an SRDF from a URDF - the MoveIt Setup Assistant.

5.4.1. SRDF and MoveIt Setup Assistant

MoveIt setup assistant window shown in Figure 5.2 can be launched with the following command:

```
$ ros2 launch moveit_setup_assistant setup_assistant.launch.py
```

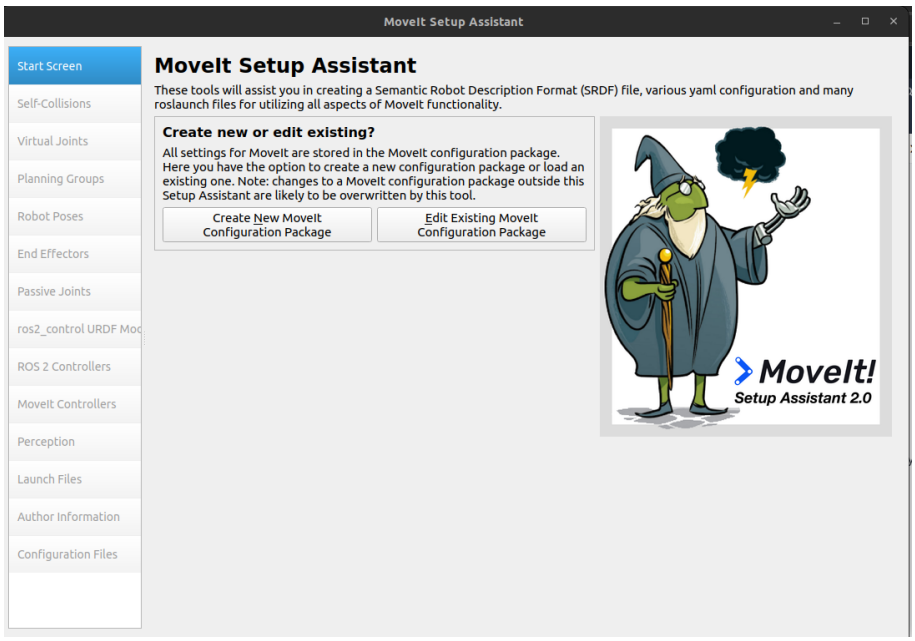


Figure 5.2. MoveIt Setup Assistant window

The configuration process is divided into a sequence of tabs on the left side of the window, which should be completed in order from top to bottom. Table 5.1 presents the purpose of the most important tabs used for the project.

Table 5.1. Most important tabs in MoveIt Setup Assistant

Tab	**Purpose**
Start Screen	Uploading and verifying URDF of the robot
Self-Collisions	Automatic generation of a self-collision matrix to reduce unnecessary collision checks during planning
Virtual Joints	Adding the connection between the robot base and the virtual world coordinates
Planning groups	Defining kinematic chain and joint groups used for motion planning
Robot Poses	Creating robot poses that can later be chosen during the operation
End Effector	Assigning a group that acts as the end effector of the robot
ros2_control URDF Modification	Generating hardware interface tags required for ros2_control integration
ROS 2 Controllers	Defining joint trajectory controllers used for executing motion
MoveIt Controllers	Assigning controllers used by MoveIt to execute planned trajectories
Launch Files	Generating a launch file for easy launch of the project in RViz
Configuration Files	Exporting the final MoveIt configuration package

Table 5.2 represents the generated configuration files with their purpose, these files form a complete MoveIt planning and execution pipeline, enabling motion planning, visualization, and simulated execution using a fake hardware interface.

The fake hardware interface provided by MoveIt allows for the validation of kinematics, joint limits, and controller configuration without requiring a physical connection to the robot. In this project, the simulated joint states generated by the fake hardware interface were used as reference commands for the real robot. These joint values were extracted from the MoveIt execution pipeline and transmitted to the physical robot controller, which executed the motion in an open-loop manner without feedback from joint encoders.

Table 5.2. Configuration files generated by the MoveIt Setup Assistant

File	Purpose
initial_positions.yaml	Defines default joint positions used by the fake hardware interface for simulation and testing in RViz.
joint_limits.yaml	Specifies position, velocity, and acceleration limits for each joint. Minor compatibility issues were encountered in the generated file, where integer values were used instead of floating-point values. These entries were manually corrected to ensure proper operation of the planning pipeline.
kinematics.yaml	Defines inverse kinematics solvers and solver parameters for each planning group.
moveit_controllers.yaml	Maps MoveIt execution pipelines to available ROS 2 controllers. Additional parameters were added manually to correctly bind the FollowJointTrajectory action interface used for trajectory execution.
moveit.rviz	Preconfigured RViz layout for visualizing the robot, planning scene, and motion planning interface.
my_robot.ros2_control.xacro	Defines the ros2_control hardware interface, including joint interfaces and controller configuration.
my_robot.srdf	Semantic robot description defining planning groups, end effectors, and disabled self-collisions.
my_robot.urdf.xacro	Assembles the complete robot description by including the kinematic and geometric URDF and combining it with ros2_control hardware interfaces and fake hardware configuration.
pilz_cartesian_limits.yaml	Defines Cartesian velocity and acceleration limits for the Pilz industrial motion planner.
ros2_controllers.yaml	Configuration of joint trajectory controllers used for motion execution.

The robot model was tested using a launch file provided by the Setup Assistant:

```
$ ros2 launch srdf demo.launch.py
```

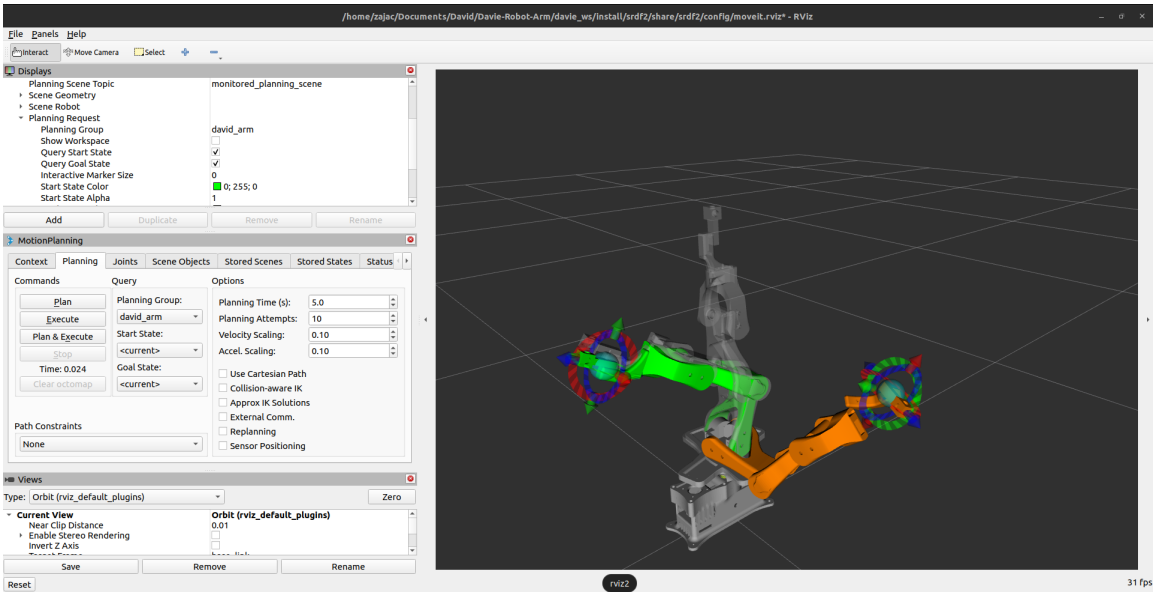


Figure 5.3. Manual testing of motion planning in RViz

The graphical environment allows for easy testing of motion planning and execution using a simulated (fake) hardware interface, which is useful for validating kinematics and configuration before deploying to real hardware. Figure 5.3 shows the robot visualized in RViz with working motion planning.

5.4.2. Trajectory planning with MoveIt

Motion planning in MoveIt can be performed using a variety of planners, which can be selected depending on the required performance and motion characteristics.

1. **Open Motion Planning Library (OMPL)** [14] — the default planner in MoveIt, providing a wide range of sampling-based planning algorithms [2].

2. **Pilz Industrial Motion Planner** [15] — designed for industrial applications, enabling linear and circular motions.

3. **Stochastic Trajectory Optimization for Motion Planning (STOMP)** [22] — an optimization-based motion planner derived from the PI^2 (Policy Improvement with Path Integrals) algorithm [1] .

4. **Search-Based Planning Library (SBPL)** [20] — a set of planners based on discrete search methods.

5. **Covariant Hamiltonian Optimization for Motion Planning (CHOMP)** [7] — an optimization-based planner suitable for smooth, collision-free trajectories.

Figure 5.4 shows how the motion planner can be chosen in RViz.

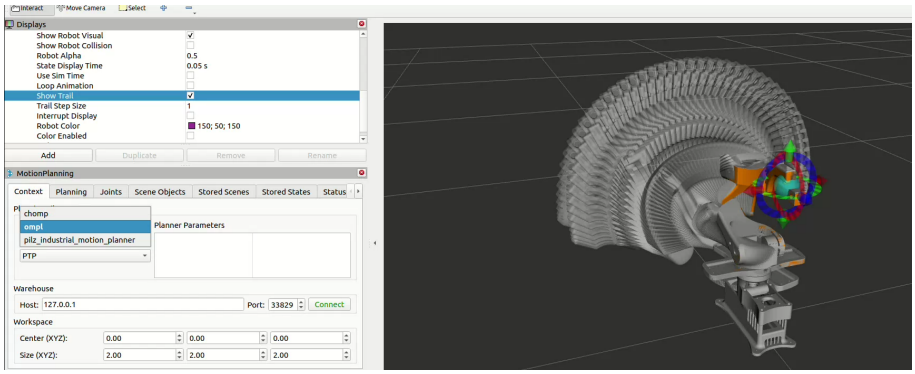


Figure 5.4. Motion planner option in the GUI, with the trail visualization enabled

5.5. Hardware interface and Micro-ROS

After implementing high-level trajectory planning using MoveIt, a hardware interface layer was required to execute the planned motions on the physical robot. This layer is responsible for translating joint-level commands generated by ROS 2 controllers into the electrical control signals required by the actuators.

5.5.1. Servomotors control via PCA9685

Servomotors are controlled using a PCA9685 PWM driver connected to the ESP32 via the I^2C interface. The ESP32 microcontroller acts as the real-time controller responsible for generating precise PWM signals for multiple servo channels.

To minimize dependencies and improve control over timing, a lightweight custom interface was implemented using direct register-level access rather than a high-level library. The PCA9685 is initialized during the system startup phase, where the PWM frequency and output channels are configured.

5.5.2. Stepper motor control and homing

Unlike servomotors, the stepper motor does not provide absolute position feedback and operates by executing a predefined number of steps. To establish a known reference position, a homing procedure was implemented.

During the homing sequence, the stepper motor rotates at low speed until a Hall-effect sensor mounted on the base detects a magnetic field generated by a permanent magnet attached to the rotating element. Once triggered, the current position is defined as the home position, allowing subsequent movements to be executed relative to this reference.

5.5.3. Integrating hardware interface with Micro-ROS

To enable direct communication between the microcontroller and the ROS 2 system, the ESP32 was integrated using the Micro-ROS framework. Micro-ROS allows microcontrollers to function as native ROS 2 nodes, publishing and subscribing to topics using standard ROS 2 message types. Listing 5.4 represents a structure of the Micro-ROS node.

```
void setup() {
  init_hardware();            // I2C, PCA9685, stepper pins, sensors
  init_micro_ros_node();      // allocator, node, executor
  create_jointstate_sub();    // JointState subscriber
}

void joint_state_callback(msg) {
  for each joint in msg:
    convert_position_to_actuator_units();
    send_command_to_hardware();
}

void loop() {
  rclc_executor_spin_some();
}
```

Listing 5.4. Logical structure of the Micro-ROS firmware running on the ESP32

On the host computer, a Micro-ROS agent is launched, which acts as a bridge between the DDS-based ROS 2 network and the serial communication interface used by the microcontroller. It can be launched with the command specifying the bitrate and the port where the device is connected:

```
$ ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB0 -b 115200
```

The Micro-ROS firmware was deployed to the ESP32 using the Arduino IDE. Special attention was required to ensure compatibility between the ROS 2 distribution used on the host system and the corresponding version of the `micro_ros_arduino` library. Mismatched versions resulted in unstable communication, which was resolved by aligning the library version with the ROS 2 Humble distribution.

5.5.4. Data exchange and verification

To verify correct communication, a test node running on the ESP32 was implemented to publish a counter value, which was successfully received and visualized on the host system using standard ROS 2 tools.

Subsequently, subscribers were implemented on the microcontroller to receive joint position commands. Joint position commands were transmitted using the standard `sensor_msgs/JointState` message, ensuring compatibility with MoveIt-generated trajectories.

5.5.5. MoveIt–Micro-ROS bridge node

Instead of directly interfacing MoveIt with the physical hardware using `ros2_control`, a custom bridge node was implemented. This node subscribes to the `/joint_states` topic generated by MoveIt using a fake hardware interface and republishes the desired joint positions to the topic Micro-ROS-based controller is subscribed to.

This approach enables open-loop execution of planned trajectories on the real robot while significantly reducing system complexity on the embedded side. The bridge node performs joint filtering, reordering, and rate limiting, ensuring that only relevant joint positions are transmitted to the microcontroller at a fixed update frequency. Listing 5.5 represents the pseudocode of bridge node.

```
initialize node
create publisher to "/esp_joint_states"
subscribe to "/joint_states"

on JointState message received:
  store latest message

every 40 ms:
  if no message received yet:
    return
  extract positions of selected joints
  reorder joints to match hardware layout
  publish filtered JointState
```

Listing 5.5. Logical structure of the MoveIt–Micro-ROS bridge node

The update period of 40 ms was selected after running tests on the system. It had to be lower than the operating frequency of the servomotors, which are driven at 50 Hz. Higher command update rates led to timing jitter and command overlap on the embedded controller, resulting in poor motion quality. The selected update rate provided smooth motion while remaining within the timing constraints of the Micro-ROS communication and actuator control loop.

5.6. Advanced control: voice interface

One of the key advantages of ROS 2 is its modularity. Robot behavior can be modified by changing individual nodes without affecting the rest of the system, provided that message interfaces remain consistent.

To demonstrate this, a voice recognition system was created to control the robot's Tool Center Point (TCP) using voice commands. The voice recognition interface is implemented as a separate ROS 2 node, allowing alternative control methods to be integrated without modifying the motion planning or hardware layers.

5.6.1. C++ API interface for MoveIt

High-level motion execution was implemented using the C++ MoveIt API. A dedicated ROS 2 node was created to interface directly with the MoveIt planning scene and execute Cartesian and joint-space motions of the robot.

To command Cartesian motion of the robot, the pose of the TCP must be expressed relative to the robot base frame. The transform between the base frame and the tool frame was verified using the `tf2` framework.

The transform was inspected using the following command:

```
$ ros2 run tf2_ros tf2_echo base_link tool_1
```

The command returned the following data:

```
- Translation: [0.000, 0.002, 0.538]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, 1.000, 0.000]
- Rotation: in RPY (radian) [0.000, -0.000, 3.142]
- Rotation: in RPY (degree) [0.000, -0.000, 180.000]
- Matrix:
 -1.000 -0.000  0.000  0.000
  0.000 -1.000  0.000  0.002
  0.000  0.000  1.000  0.538
  0.000  0.000  0.000  1.000
```

This matrix encodes both the position and orientation of the TCP and serves as the basis for Cartesian motion planning using MoveIt. Using the MoveIt C++ interface, motion commands were generated by specifying target poses for the TCP. The planning pipeline computes a collision-free trajectory, which is subsequently executed in simulation and forwarded to the hardware interface.

5.6.2. Voice recognition using the Vosk library

ROS 2 nodes can be implemented in different programming languages as long as they communicate using compatible message types. This feature was utilized by implementing a voice recognition node in Python and integrating it with a C++ motion control node. The Python node uses the offline speech recognition library Vosk [25] to detect predefined voice commands. Upon recognition, motion commands are published to a ROS 2 topic.

Instead of defining a custom message type, the standard `geometry_msgs/Twist` message was used. It consists of two three-dimensional vectors describing linear and angular motion, making it suitable for intuitive Cartesian control of the TCP. The C++ node subscribes to this topic and translates received commands into corresponding MoveIt motion goals. Figure 5.5 shows the running voice detection node and the output of the detected commands. The node detects a robot command if the first word is "david". Once activated, the command is parsed into two components: the desired motion direction (e.g., *"go up"*, *"go left"*, or *"pitch"*) and a numerical value defining the magnitude of the motion applied to the TCP.

```
[INFO] [1766829068.230976661] [voice_commands_node]: Voice control node started. Say: 'david go up 10'
[INFO] [1766829077.597056006] [voice_commands_node]:  Recognized: this is voice command test
[INFO] [1766829084.073311326] [voice_commands_node]:  Recognized: voice recognition is working well
[INFO] [1766829090.783093249] [voice_commands_node]:  Recognized: let's try using the first voice command
[INFO] [1766829095.942870678] [voice_commands_node]:  Recognized: david go up ten
[INFO] [1766829095.943490081] [voice_commands_node]: Published command: geometry_msgs.msg.Twist(linear=ge
ometry_msgs.msg.Vector3(x=0.0, y=0.0, z=10.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
[INFO] [1766829103.313123359] [voice_commands_node]:  Recognized: david go left twenty
[INFO] [1766829103.313634694] [voice_commands_node]: Published command: geometry_msgs.msg.Twist(linear=ge
ometry_msgs.msg.Vector3(x=0.0, y=-20.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0))
[INFO] [1766829108.787918449] [voice_commands_node]:  Recognized: david pitch fifty
[INFO] [1766829108.788713553] [voice_commands_node]: Published command: geometry_msgs.msg.Twist(linear=ge
ometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=50.0, z=0.0))
```

Figure 5.5. Voice recognition node sending twist messages when a command is detected.

5.7. Software summary

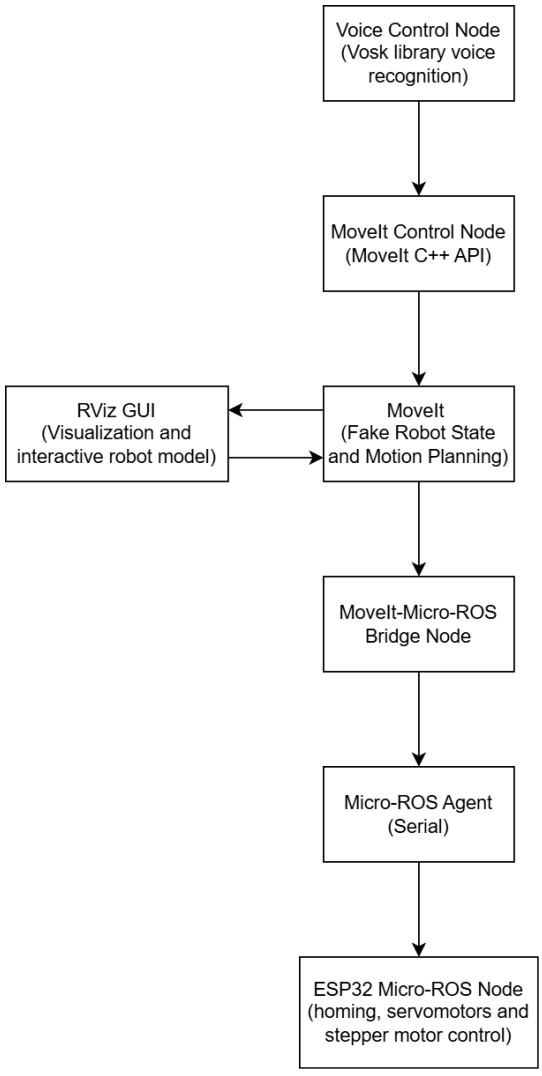


Figure 5.6. Simplified Software Architecture

The final software system consists of several modular ROS 2 nodes cooperating within a unified architecture.

The robot TCP position can be set using the GUI through RViz by manually dragging the robot to a desired position or selecting one of the predefined positions.

An alternative control mode is provided through voice commands. A Python-based voice recognition node detects spoken commands and publishes motion requests. Commands are received by a C++ node that interfaces with the MoveIt API and generates motion trajectories. Joint targets from the simulated robot are then forwarded to the bridge node, which transmits the joint states to the ESP32 microcontroller using Micro-ROS. The microcontroller executes the commands and provides the hardware interface for the actuators.

Figure 5.6 presents the finished software architecture of the project, with a clear separation of perception, motion planning, and hardware control, enabling each component to be modified or replaced independently.

6. Conclusion

This thesis presented the process of designing a complete robotic arm system, integrating mechanical construction, electrical design, and a distributed control architecture based on ROS 2.

The construction of the robot demonstrated an interesting use of 3D printing technology and the construction of a robot arm using a variety of actuator types.

The project demonstrates a separation between high-level motion planning and low-level real-time control. Computationally demanding tasks such as trajectory planning, visualization, and user interaction were executed on a PC running ROS 2, while time-critical actuator control was delegated to an ESP32 microcontroller integrated using micro-ROS.

This architecture allowed for reliable motor control while maintaining flexibility at the software level.

The main technical achievements of the project include:

- Design and construction of a multi-axis robotic arm with mixed actuation, including servomotors and a stepper-driven base.
- Development of a custom electrical system with multiple power domains and appropriate voltage regulation for high-current actuators.
- Integration of the robot into the ROS 2 ecosystem by creating URDF and SRDF descriptions, enabling visualization and collision-aware motion planning.
- Successful configuration and use of MoveIt 2 for trajectory planning and execution.
- Implementation of a hardware interface using micro-ROS, allowing the ESP32 microcontroller to function as a ROS 2 node.
- Demonstration of multi-language node system through the integration of Python-based voice recognition with C++ motion planning API.

The layered architecture proved to be well suited for the robotic system, combining real-time hardware constraints with complex planning requirements. Delegating actuator control to a microcontroller ensured stable signal generation and predictable behavior, while ROS 2 provided a powerful framework for modular software development. The use of standard ROS 2 tools and message types significantly simplified debugging, visualization, system integration, and allowed for simple reusability of the project in different configuration. The project confirms that micro-ROS is a viable solution for embedding microcontrollers into modern robotic systems without sacrificing architectural consistency.

Despite its successful implementation, the system has several limitations. The robot relies on open-loop control without direct position or torque feedback from the actuators. This limits positioning accuracy and makes the system sensitive to mechanical backlash and load variations.

The developed system provides a solid foundation for future upgrades. Possible directions for further work include:

- Integration of joint-level feedback using encoders to enable closed-loop position or torque control.
- Expansion of the perception pipeline using cameras or depth sensors for object detection.
- Addition of a gripper for executing pick-and-place tasks.
- Deployment of safety mechanisms such as workspace limits, collision monitoring, and emergency stop handling.
- Increasing rigidity by using a stronger filament for 3D prints.

In conclusion, the project demonstrates that modern robotic systems can be effectively developed using open-source tools and modular design principles. The presented solution combines theoretical knowledge with practical engineering, resulting in a functional and extensible robotic platform.

The system not only fulfills the initial design goals but also provides a scalable base for further research and development in robotic control and human–robot interaction.

Bibliography

[1] Evangelos Theodorou, Jonas Buchli, and Stefan Schaal. „*Learning Policy Improvements with Path Integrals*". University of Southern California. 2010.

[2] Ioan A. Şucan, Mark Moll, and Lydia E. Kavraki. „*The Open Motion Planning Library*". In: *IEEE Robotics & Automation Magazine* 19.4 (Dec. 2012). *https://ompl.kavrakilab.org*, pp. 72–82. DOI: *10.1109/MRA.2012.2205651*.

[3] Steven Macenski et al. „*Robot Operating System 2: Design, architecture, and uses in the wild*". In: *Science Robotics* 7.66 (2022), eabm6074. DOI: *10.1126/scirobotics.abm6074*.

[4] „*3D Printer Filament Types and 3D Printing Guide*". All3DP. URL: *https://all3dp.com/1/3d-printer-filament-types-3d-printing-3d-filament/* (visited on 2025-12-04).

[5] „*AR4 - "The Robot Anyone Can Build"*". Annin Robotics. URL: *https://anninrobotics.com/* (visited on 2025-01-15).

[6] „*BCN3D MOVEO: A Fully Open Source 3D Printed Robot Arm*". BCN3D. URL: *https://bcn3d.com/bcn3d-moveo-the-future-of-learning-robotic-arm/* (visited on 2025-12-13).

[7] „*CHOMP (Covariant Hamiltonian Optimization for Motion Planning)*". Nathan Ratliff. URL: *https://www.nathanratliff.com/thesis-research/chomp* (visited on 2025-12-20).

[8] „*Creating URDF File*". Open Source Robotics Foundation. URL: *https://wiki.ros.org/urdf#Creating_URDF* (visited on 2025-12-03).

[9] „*fusion2urdf Add-in for Autodesk Fusion 360*". syuntoku14. URL: *https://github.com/syuntoku14/fusion2urdf* (visited on 2025-12-04).

[10] „*Mech Arm 270 from Elephant Robotics*". Elephant Robotics. URL: *https://www.elephantrobotics.com/en/support-mecharm-270-en/* (visited on 2025-01-15).

[11] „*Micro-ROS: ROS 2 for Microcontrollers*". Micro-ROS Contributors. URL: *https://micro.ros.org/* (visited on 2025-01-15).

[12] „*Micro-ROS: Source Code and Libraries*". Micro-ROS Contributors. URL: *https://github.com/micro-ROS* (visited on 2025-01-15).

[13] „*MoveIt Main Website*". PickNik Robotics. URL: *https://moveit.ai/* (visited on 2025-10-05).

[14] „*OMPL (The Open Motion Planning Library)*". Rice University. URL: *https://ompl.kavrakilab.org/* (visited on 2025-12-20).

[15] „*Pilz Industrial Motion Planner*". PickNik Robotics. URL: *https://moveit.picknik.ai/main/doc/how_to_guides/pilz_industrial_motion_planner/pilz_industrial_motion_planner.html* (visited on 2025-12-20).

[16] „*ROS 2 Design Articles*". Open Source Robotics Foundation. URL: *https://design.ros2.org/articles/changes.html* (visited on 2025-12-03).

[17] „*ROS2 Common Message Type*". ahcorde. URL: *https://github.com/ros2/common_interfaces* (visited on 2025-12-04).

[18] „*ROS2 Contributors and Governance*". Open Robotics. URL: *https://docs.ros.org/en/humble/The-ROS2-Project/Governance.html* (visited on 2025-12-10).

[19] „*RViz User Guide*". Open Robotics. URL: *https://docs.ros.org/en/humble/Tutorials/Intermediate/RViz/RViz-User-Guide/RViz-User-Guide.html* (visited on 2025-04-23).

[20] „*Search-Based Planning Library (SBPL)*". Open Robotics. URL: *https://wiki.ros.org/sbpl* (visited on 2025-12-20).

[21] „*Semantic Robot Description Format (SRDF) Documentation*". Open Source Robotics Foundation. URL: *https://wiki.ros.org/srdf* (visited on 2025-12-03).

[22] „*STOMP Motion Planner (Stochastic Trajectory Optimization for Motion Planning)*". Open RoboticsOpen Robotics. URL: *https://wiki.ros.org/stomp_motion_planner* (visited on 2025-12-20).

[23] „*Unified Robot Description Format (URDF) Documentation*". Open Source Robotics Foundation. URL: *https://wiki.ros.org/urdf* (visited on 2025-12-03).

[24] „*URDF XML Specification*". Open Source Robotics Foundation. URL: *https://wiki.ros.org/urdf/XML* (visited on 2025-12-03).

[25] „*VOSK Voice Recognition Toolkit*". Alphacephei. URL: *https://alphacephei.com/vosk/L* (visited on 2025-12-04).